

Conversational Search Agent — End-to-End Pipeline Summary

TechJam 2026 • Conversational E-Commerce Search • generated 2026-08-29

Headline. The agent finds the hidden target product with no LLM, no network and only the Python standard library. Verified locally on the 200-session public set:

Metric	Value	Baseline
Hit Rate@10	0.940	0.125
MRR	0.791	0.068
MTTC (mean turns to hit)	3.41	9.81
Efficiency	0.760	0.119
Recommended Technical Score	0.8592	0.1067

Score reproduced exactly by re-running `python3 -m evaluator.local_evaluator` against the unmodified organizer evaluator (0.859229). 29/29 tests pass.

1. Consistency with IMPLEMENTATION.pdf

Verdict: the agent's behaviour matches IMPLEMENTATION.pdf. Every shipped constant, stage boundary, and headline/per-scenario metric in the document reproduces against the current code. One module — the intent router — has diverged in source, but not in observable behaviour.

Area	Documented	In current code	Match?
S1 index	FTS5, 7 cols, weights (0,6,4,2.5,2.5,1.5,1); cached; stopword+boilerplate lists	Identical (src/index.py:25, src/text.py)	Yes
S3 dialog state	Accumulate turns; PRE_OVERRIDE_WEIGHT=0.35; down-weight not erase; dead_attributes; productive_turns	Identical (src/state.py:38)	Yes
S4 policy	FixedPolicy ships; InfoGainPolicy present, coverage x gain_ratio x answerability; broad_yield=6.0	Identical (src/policy.py)	Yes
S5 retrieval	terms + focused routes; RRF k=60; weight_focused=0.8; pool 300; phrase route lives in rerank	Identical (src/retrieval.py)	Yes
S6 rerank	span coverage + normalised retrieval + 0.02 popularity; length_bonus 0.12; depth 200	Identical (src/rerank.py)	Yes
S7 timing	hold until turn 3; confidence margin 0.20; earliest turn 2	Identical (starter/agent.py, AgentConfig)	Yes
Metrics (all + per scenario)	0.8592; boundary 1.00 / browsing 0.963 / buying 0.938 / intent_override 0.867	Reproduced to 6 d.p.	Yes
S2 intent router	"Scoped down to phrasing only." ~68 lines: two cue regexes, two tone strings	~278 lines (merged branch feat/sensitive-intent-router): FACET_PATTERNS, confidence calibration, scenario_hint, suggested_first_recommend_turn, detect_turn_intent()	Module: NO. Behaviour: yes

Why the router divergence does not affect results. `starter/agent.py` calls `classify(state.opening)` but consumes only `route.tone` (`agent.py:120`). Every richer field the merged router computes — `confidence`, `scenario_hint`, `suggested_first_recommend_turn` — is discarded, and `detect_turn_intent()` has no caller. `route.tone` only prefixes the English message, which the simulated customer never reads (it keys on `ask_attribute` alone). The two tone strings are unchanged from the documented version, so the enlarged router is inert: the evaluator score is bit-for-bit what `IMPLEMENTATION.pdf` reports.

Minor staleness: the document says "25 tests"; the merged branch brought the suite to 29. Documented dead code is still present (search_phrases() at src/index.py:119; FacetStore built at agent.py:75 but unused at runtime under the default FixedPolicy).

2. What the agent is solving

- 50,000-product clothing/shoes/jewellery catalog. One hidden **target** the agent must surface.
- A session is ≤ 10 turns. Each turn: customer message in, agent replies with a sentence, optionally an ask_attribute, optionally ≤ 10 parent_asins.
- Session ends the instant the target appears in a shown list — **rank and turn freeze there**.
- The simulated customer discloses a new hidden constraint **only when the reply sets** ask_attribute (max 2 per answered question; exactly 4 hidden per session).
- A raised exception is silently scored as a lost session.

Three consequences drive the whole design: (1) never show an uncertain list early — a bad frozen rank is unrecoverable; (2) wrong recommendations are free, only turns cost; (3) asking a question is the only way to learn anything. Fixing just (3) — ask, and remember the answers — took the score from 0.107 to 0.781 before any retrieval tuning.

3. Pipeline at a glance

```
reset(session_id, user_profile) -> DialogState(session_id, profile)

respond(session_id, user_message, turn, top_k):

    S3 state.observe(turn, text)          append utterance (+provenance weight)
    |                                     turn 1 -> state.opening
    |                                     reversal cue -> apply_override(): all weights := 0.35
    |                                     "no preference" -> dead_attributes.add(last asked)
    |                                     new constraint span -> productive_turns += 1
    v
    S2 classify(state.opening) -> Route      (only route.tone is used downstream)
    v
    S5 retrieve(index, state, cfg)
    terms route : search_terms(full_text)      FTS5 bm25, weights (0,6,4,2.5,2.5,1.5,1), <=300
    focused route : search_terms(focused_text) only if an override fired, RRF weight 0.8
    fuse         : score += w / (60 + pos + 1) -> sort desc -> top 300
    v
    S6 rerank(index, state, pool, cfg)        top 200 only; needs >=1 constraint span
    score = 1.0*span_coverage + 1.0*(bm25 / top_bm25) + 0.02*popularity
    span_coverage = sum over disclosed spans present in product.text of (1 + 0.12*words)
    v
    S4 policy.select(state, pool)              FixedPolicy: ask "other" every turn;
    |                                         fall through (feature,use_case,style,material,color,size,category)
    |                                         skipping any attribute in dead_attributes
    v
    S7 agent._shortlist(pool, turn, top_k)
    turn < 3 and not _confident(pool) -> return [] (hold)
    _confident: (best - runner_up)/best >= 0.20 and turn >= 2
    else -> top min(10, top_k) parent_asins
    v
    message = route.tone + policy.question(attribute)
    return {message, ask_attribute, recommendations, usage:{0,0}}

(entire path wrapped in try/except -> valid empty envelope on any error)
```

4. Stage-by-stage

S1 — Catalog index (src/index.py, src/text.py)

- One pass over `catalog.jsonl` builds: an in-memory SQLite **FTS5** index (7 columns, per-field bm25 weights favouring title/categories), and a trimmed per-product record (title, categories, brand, price, ratings, and a punctuation-stripped token-joined text blob).
- The normalised text blob is what lets the reranker substring-match customer phrases like “100 leather” against “100% Leather”.
- Two removal lists before querying: conversational **stopwords** and Amazon **boilerplate** (“imported”, “machine wash”...) that saturates the catalog and adds cost without signal.
- `load_index()` caches one index per catalog path (~4s build, ~282 MB), so repeated Agent construction in the sweep harness is free.

S2 — Intent router (src/router.py)

- Classifies the opening message as **buying** vs **browsing** from linguistic cues (not template strings, which would overfit the local evaluator's fixed templates). Unknown phrasing defaults to browsing.
- Drives **only the phrasing** of the clarification question: “To narrow this down...” vs “To point you in the right direction...”. Routing the candidate pool by intent was measured and dropped (dev unchanged, holdout -0.002).
- *Divergence noted in §1*: the merged feat/sensitive-intent-router branch expanded this module with facet-density scoring, confidence calibration and a multi-turn tracker, none of which the agent wires in. Behaviour and score are unchanged.

S3 — Dialog state (src/state.py) — the stage that carries the score

- Stores every customer utterance with **provenance** (turn number + weight). Exposes `full_text()` (all turns, max recall) and `focused_text()` (weight ≥ 1.0 turns only, max precision).
- **Intent override**: on a reversal cue, pre-override utterances are **down-weighted to 0.35, not erased**. In this evaluator the “discarded” preference is still drawn from the target's own metadata, so erasing it throws away real signal. (Deliberate, documented deviation from the brief's “slot erasure”.)
- `dead_attributes`: an attribute the customer declined (“no preference for material”) is never asked again.
- `productive_turns`: whether an answer actually disclosed something new.
- Measured effect: $0.1067 \rightarrow 0.7811$.

S4 — Clarification policy (src/policy.py)

- FixedPolicy **ships**: ask the broadest question (“anything else that matters?”) every turn, falling through a ladder as attributes are declined. Wins on the untouched holdout split.
- InfoGainPolicy is retained and selectable: scores each attribute as $\text{coverage} \times \text{gain_ratio} \times \text{answerability}$ over the live candidate pool. `gain_ratio` ($\text{entropy} / \log_2$ of distinct values) fixes a real bug where raw entropy made the agent always ask about brand. It finds the target slightly more often but ranks it worse, so it does not ship.
- The attribute is placed in `ask_attribute` — the only channel that makes the customer disclose anything.

S5 — Retrieval (src/retrieval.py)

- **terms route**: bag-of-words OR query over the whole conversation — high recall. Measured 80/80 target recall at median rank 1, which is why a planned dense-vector route was cancelled: retrieval was never the bottleneck.
- **focused route**: same query over post-override turns only; runs only after an override, giving the reversal a clean channel.
- Combined with **Reciprocal Rank Fusion** ($\text{score} += \text{weight} / (60 + \text{position} + 1)$) — uses positions not scores, so routes on incomparable scales combine without calibration and an empty route just contributes nothing.

S6 — Reranking (src/rerank.py) — second-largest win

- The customer's disclosed constraints are **copied verbatim** from the target's own metadata. So the dominant signal is: does each candidate's text **literally contain** the phrases the customer used?
- $\text{score} = \text{span_coverage} + (\text{bm25} / \text{top_bm25}) + 0.02 \times \text{popularity}$, applied to the top 200 of the pool. Longer spans score slightly higher (rarer = stronger evidence).
- Popularity is pinned near zero on purpose: the target is one person's purchase, not a bestseller, so a strong popularity prior drags ranking toward famous products.
- Verbatim matching was first tried as a *retrieval* route and lost badly (47/80 recall). The same signal as a *ranking* signal over an already-good pool is pure gain: 0.7799 → 0.8543.

S7 — Recommendation timing (starter/agent.py)

- **Hold until turn 3** — no list on turns 1–2 when little is disclosed (turn 2 = too eager, turn 4 = wastes evidence; measured).
- **Confidence gate**: emit earlier when the leader is clearly ahead — $(\text{best} - \text{runner_up}) / \text{best} \geq 0.20$. Improves MTTC with MRR unchanged. 0.20 sits mid-plateau rather than at a split's noisy argmax.
- 0.8543 → 0.8592, MTTC 3.58 → 3.41.

S9 — Robustness (starter/agent.py, tests/)

- `respond()` cannot raise: the whole path is wrapped and any exception degrades that turn to a valid empty response. Each stage also degrades locally (bad FTS expression → [], illegal attribute → other, missing session → recreated).
- 29 tests: contract validation + hostile input; each stage in isolation (incl. an assertion that the gain-ratio bug cannot return); an end-to-end regression test that fails if the holdout score drops below 0.78.

5. Results (verified 2026-08-29)

Split / scenario	Sessions	Hit@10	MRR	MTTC
Public (all)	200	0.940	0.791	3.41
boundary	10	1.000	0.846	4.00
browsing	80	0.963	0.811	3.08
buying	80	0.938	0.794	3.21
intent_override	30	0.867	0.713	4.60

Intent override is the weakest scenario and the clearest target for further work: part of the gap is structural (those sessions cannot convert before the reversal arrives on turn 3–4), the rest is the never-swept `PRE_OVERRIDE_WEIGHT = 0.35` and `weight_focused = 0.8`.

Operational	Value
Model / API	None. No LLM, no network, no credentials.
Dependencies	Python standard library only
Cost / tokens	\$0.00 / 0 prompt, 0 completion
Index build	~4 s once at startup (~282 MB peak)
Per-turn latency	~17 ms mean, ~20 ms p95
Full 200-session evaluation	~30–37 s

6. Highest-value next steps (from IMPLEMENTATION.pdf §10)

- **Learn the ~8 hand-set weights** (field weights, RRF weights, override weight, rerank weights, confidence margin) with one offline logistic-regression pass over the 176 known-correct public sessions. Highest expected value, stays

fully offline.

- **Wire a facet-agreement signal into the reranker.** `src/facets.py` already extracts material/colour/size/brand/price/category for every product but no ranking signal reads it; targets MRR (30% of score).
- **Category anchoring in rerank:** boost candidates whose category matches the opening message — directly attacks the traced “generic leather belt at rank 15” failure.
- **Close the ask→rank loop:** when the top two candidates differ only on colour, ask about colour specifically. Connects S4/S6/S7 into a feedback loop instead of a one-way pass.
- **Tidy the router:** either wire in the merged branch's confidence / scenario_hint outputs (and re-measure), or revert it to the documented ~68-line version so source matches IMPLEMENTATION.pdf.

Source of record: IMPLEMENTATION.md / .pdf, ARCHITECTURE.md, starter/agent.py, src/*.py. Evaluator (evaluator/local_evaluator.py) and public labels confirmed unchanged from the organizer release.